Dossier de Modélisation UML : Agence de Voyage de Cotonou

Conception orientée objet, dictionnaire des classes, associations, multiplicités et diagramme PlantUML

I. Choix d'Architecture Orientée Objet

La conception de l'application console en C++ s'articule autour d'une architecture modulaire à **6 classes**, respectant les principes d'encapsulation et de séparation des responsabilités :

- **5 classes entités (métier) : Passager**, **Vol**, **Reservation**, **Paiement** et **Annulation**, qui modélisent les objets fondamentaux du domaine.
- **1 classe contrôleur : AgenceVoyage**, qui centralise les collections dynamiques (**std::vector**), applique les règles de gestion et orchestre les interactions avec l'utilisateur.

II. Identification des Classes, Attributs et Méthodes

Conformément aux règles d'encapsulation UML, tous les attributs sont déclarés en visibilité privée (**-**) et manipulés par des méthodes publiques (**+**).

Classe	Attributs (Privés -)	Méthodes Principales (Publiques +)
Passager	- idPassager : int - nom : string - prenom : string - telephone : string	+ Passager(id, nom, prenom, tel) + getIdPassager() : int + getNom() : string + getPrenom() : string + afficherDetails() : void
Vol	- numeroVol : string - destination : string - dateDepart : string - capacite : int - placesDisponibles : int - prixBillet : double	+ Vol(num, dest, date, cap, prix) + getNumeroVol() : string + getPlacesDisponibles() : int + getPrixBillet() : double + decrementerPlaces() : void + incrementerPlaces() : void + afficherDetails() : void
Reservation	- idReservation : int - statut : string *(En attente, Payée, Annulée)* - passager : Passager* - vol : Vol*	+ Reservation(id, p, v) + getId() : int + getPassager() : Passager* + getVol() : Vol* + getStatut() : string + setStatut(s : string) : void + afficherDetails() : void

Classe	Attributs (Privés -)	Méthodes Principales (Publiques +)
Paiement	- idPaiement : int - montant : double - modePaiement : string - reservation : Reservation*	+ Paiement(id, res, montant, mode) + effectuerPaiement() : void
Annulation	- idAnnulation : int - motif : string - statutRemboursement : string - reservation : Reservation*	+ Annulation(id, res, motif)
AgenceVoyage *(Contrôleur)*	- passagers : vector<Passager> - vols : vector<Vol> - reservations : vector<Reservation> - paiements : vector<Paiement> - annulations : vector<Annulation>	+ enregistrerPassager() : void + ajouterVol() : void + afficherVolsDisponibles() : void + effectuerReservation() : void + effectuerPaiement() : void + annulerReservation() : void + consulterReservations() : void

III. Associations et Multiplicités

Association	Multiplicité	Justification Métier
Passager — Reservation	**1** à **0..***	Un passager enregistré peut effectuer zéro ou plusieurs réservations. Une réservation appartient à un et un seul passager.
Vol — Reservation	**1** à **0..***	Un vol peut recevoir plusieurs réservations dans la limite de sa capacité. Une réservation concerne un unique vol.
Reservation — Paiement	**1** à **0..1**	Une réservation est soit en attente (0 paiement), soit réglée une unique fois (1 paiement).
Reservation — Annulation	**1** à **0..1**	Une réservation active n'a pas d'annulation (0), ou fait l'objet d'une seule annulation définitive (1).
AgenceVoyage o— Entités	**1** à *	Relation d'agrégation : le gestionnaire centralise et administre l'ensemble des instances créées en mémoire.

IV. Diagramme de Classes (Code Source PlantUML)

Conformément aux exigences du cahier des charges, le code source suivant permet de générer directement le diagramme de classes sous **PlantUML** :

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam class {
    BackgroundColor #F8FAFC
    BorderColor #0F2942
    ArrowColor #0F2942
    HeaderBackgroundColor #E0F2FE
}

title Diagramme de Classes UML - Agence de Voyage de Cotonou

class AgenceVoyage {
    - passagers : vector<Passager>
    - vols : vector<Vol>
    - reservations : vector<Reservation>
    - paiements : vector<Paiement>
    - annulations : vector<Annulation>
    + enregistrerPassager() : void
    + ajouterVol() : void
    + afficherVolsDisponibles() : void
    + effectuerReservation() : void
    + effectuerPaiement() : void
    + annulerReservation() : void
    + consulterReservations() : void
}

class Passager {
    - idPassager : int
    - nom : string
    - prenom : string
    - telephone : string
    + Passager(id : int, nom : string, prenom : string, tel : string)
    + getIdPassager() : int
    + getNom() : string
    + getPrenom() : string
    + afficherDetails() : void
}

class Vol {
    - numeroVol : string
    - destination : string
    - dateDepart : string
    - capacite : int
    - placesDisponibles : int
    - prixBillet : double
    + Vol(num : string, dest : string, date : string, cap : int, prix : double)
    + getNumeroVol() : string
    + getPlacesDisponibles() : int
    + getPrixBillet() : double
    + decrementerPlaces() : void
    + incrementerPlaces() : void
    + afficherDetails() : void
}

class Reservation {
    - idReservation : int
    - statut : string
    + Reservation(id : int, p : Passager*, v : Vol*)
    + getId() : int
    + getPassager() : Passager*
```